# Google Pay™ Acceptance via SimplePay

Merchant Documentation

*General Information*

*01.09.2026.*

# TABLE OF CONTENTS


1 Overview ..... 4

2 Google Pay Payment Flow ..... 5

2.1 Customer Interaction – Pressing the Google Pay Button ..... 6

2.2 Google Pay Payment Frontend ..... 6

2.2.1 Google Pay Data ..... 7

2.2.2 SimplePay Data ..... 7

2.2.3 Merchant System Data ..... 7

2.2.4 Transaction Data ..... 7

2.2.5 Authorization Methods ..... 8

2.2.6 Required Google Pay API Configuration and Initialization ..... 9

2.2.7 Billing Address Parameters ..... 11

2.2.8 Required fields sent from the frontend to the merchant backend: ..... 11

2.2.9 Google Pay Payment Backend, doGooglePay ..... 12

2.2.10 Invoking the Google Pay payment sheet ..... 14

2.3 Supported Payment Methods ..... 15

3 Merchant Requirements Mandated by Google ..... 15

3.1 Business Compliance ..... 15

3.2 Prohibited and Restricted Activities ..... 16

4 Google Pay Approval-Related Elements ..... 16

4.1 Google Pay Merchant Identification ..... 16

4.1.1 Obtaining the gpayMerchantId (Google Pay merchantId) – Creating a Merchant Profile (Google Pay & Wallet Console) ..... 16

4.2 Domain and Application Compliance ..... 17

4.2.1 Domain verification ..... 17

4.2.2 Submitting a Website (Domain) and Requesting Approval (for Production Use) 17

4.3 What We Require from the Merchant Regarding the merchantId ..... 18

4.3.1 Submission process ..... 18

5 Technical and System-Level Elements Provided by SimplePay ..... 18

6 Testing and Production ..... 19

7 Summary ..... 20

## Document History

| Date | Version | Change |
|---|---|---|
| 03.03.2026. | 260303 | Initial release |
| 06.03.2026. | 260306 | Fix Google Pay comments |
| 11.03.2026. | 260311 | Fix Google Pay comments |
| 16.03.2026. | 260316 | Fix Google Pay comments |
| 04.05.2026. | 260504 | Corporate branding updates |
| 01.09.2026. | 260901 | Wording changes based on developer comments |

# 1 Overview

SimplePay is the online payment solution of SimplePay Plc. This documentation describes the Google Pay acceptance via SimplePay feature, based on the SimplePay merchant API and transaction management v2.

As such, the following sections cannot be interpreted without knowledge of the general SimplePay payment documentation!

The current version of the general SimplePay technical documentation is available in the download section of the merchant admin interface (on both the sandbox and production systems), or at the following URLs:

- **Hungarian:** https://simplepartner.hu/download.php?target=v21dochu
- **English:** https://simplepartner.hu/download.php?target=v21docen

This documentation references or assumes familiarity with the above general documentation when discussing specific topics.

Google Pay™ is a digital payment solution that enables customers to complete card payments quickly and securely on web interfaces and in mobile applications. During the payment process, card data is processed in tokenized form, therefore the actual card details are never transmitted to the merchant's system.

Merchants implementing Google Pay must comply with the official Google Pay brand and integration requirements. The official documentation is available at:

**Google Pay Web developer documentation:**

https://developers.google.com/pay/api/web/overview

**Google Pay Web Brand Guidelines:**
https://developers.google.com/pay/api/web/guides/brand-guidelines

**Google Pay Web Integration Checklist:**
https://developers.google.com/pay/api/web/guides/test-and-deploy/integration-checklist

If the payment does not require 3DS authentication, the customer remains on the merchant's payment initiation page and SimplePay does not initiate a browser redirect or back call. If the card issuer requires a 3DS challenge, the customer must be redirected to the redirectUrl returned by the doGooglePay endpoint. After authentication, SimplePay redirects the customer back to the URL specified by the merchant in the url field of the doGooglePay request.

From the customer's perspective, the process on a mobile device consists of the following: at checkout, when selecting payment methods, the customer taps the Google

Pay button, selects which card saved in the Google system to use for payment, and initiates the transaction. If no 3DS challenge is required, all subsequent processing takes place in the background and the customer remains on the payment initiation page.

SimplePay, as a payment technology provider, enables Google Pay acceptance through **its own API interface**. This means that merchants do not need to establish a direct technical connection with the Google Pay system.

The purpose of this document is to describe **which requirements and approvals are needed from Google**, and which areas are handled within the SimplePay service.

# 2 Google Pay Payment Flow

The payment flow consists of the following steps:

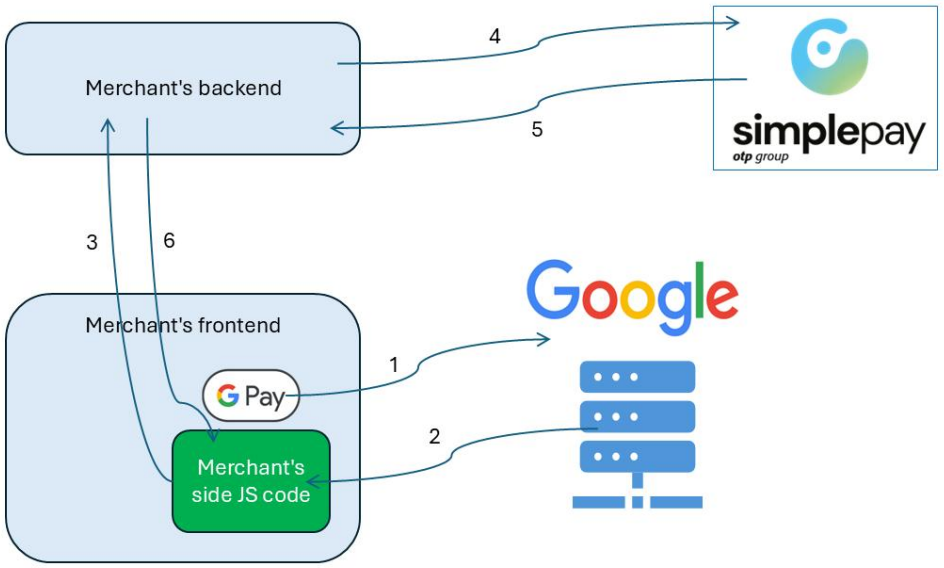
- the customer selects products in the merchant's system, which determines the final amount to be paid
- a JavaScript frontend is used to initiate the payment, displaying a button that complies with Google's requirements
- when the customer presses the button, a JavaScript function executes, which registers the transaction through the Google API and returns the token that must be sent to the SimplePay system
- the API returns a response indicating the result to the merchant's system
- The synchronous response from the doGooglePay endpoint is received by the merchant backend, which returns the required result data to the frontend.
- If no 3DS challenge is required, the customer remains on the payment initiation page. The merchant frontend displays the result on the same page or optionally performs a merchant-controlled redirect to a result page.
- If the response contains error code 3004 in the errorCodes array, the frontend redirects the browser to the returned redirectUrl. After the 3DS authentication has been completed, SimplePay redirects the customer to the URL provided in the url field of the original doGooglePay request.
- Order fulfilment should be based on the IPN message or a server-side transaction status query. Neither the synchronous API response nor the browser back call should be treated as the sole authoritative confirmation of payment.

## 2.1 Customer Interaction – Pressing the Google Pay Button

When the customer clicks the **Google Pay button** on the payment page, a Google-provided interface appears in the browser, where the customer can select a payment method (e.g., a previously saved card) or enter new card details.

**In this step, the following occurs:**

- the Google Pay interface opens,
- the customer selects a payment method,
- the customer's payment data (tokenized) is retrieved from the Google server (detailed in the developer documentation).




## 2.2 Google Pay Payment Frontend

The merchant's system has several options for implementing the payment frontend. The SimplePay SDK provides a sample implementation.

However, the frontend can be implemented in multiple ways within the merchant's system. The developer can find further information on the Google developer site: https://developers.google.com/pay/api/web/guides/tutorial

The following sections illustrate the implementation based on the sample JavaScript included in the SDK, which represents Google's minimum recommendation.

To simplify the merchant's implementation, the variables required for the minimum implementation have been extracted into the **simplePayTransaction** object in the JavaScript.

**Structure of the simplePayTransaction object:**

In the sample implementation, the parameters required for Google Pay initialization are grouped in a helper object called **simplePayTransaction**.

```
const simplePayTransaction = {
  gpayMerchantId: "YOUR_GPAY_MERCHANT_ID",
  gpayMerchantName: "Merchant name",
  gpayEnvironment: "TEST",
  gateway: "simplepay",
  gatewayMerchantId: "YOUR_SIMPLEPAY_MERCHANT_ID",
  authorizationEndpoint: "/googlepay/authorize",
  notificationPage: "/payment-result",
  paymentType: "CARD",
  total: "100.00",
  currency: "HUF"
}
```

### 2.2.1 Google Pay Data

- **gpayMerchantId:** the merchant's identifier in the Google Pay system
- **gpayMerchantName:** the merchant's name in the Google Pay system
- **gpayEnvironment:** the Google Pay environment currently in use (TEST, PROD)

### 2.2.2 SimplePay Data

- **gateway:** payment gateway; constant value **„simplepay"**
- **gatewayMerchantId:** merchant account identifier in the SimplePay system

### 2.2.3 Merchant System Data

- **authorizationEndpoint**: the endpoint in the merchant's system where the frontend JavaScript forwards the Google Pay token and payment data
- **notificationPage:** an optional merchant result page. In a non-3DS flow, the merchant frontend may navigate to this page after processing the synchronous doGooglePay response. This is not a SimplePay back URL and SimplePay does not call it automatically. The return URL used after a 3DS challenge is not this frontend property. It is the url field configured by the merchant backend in the doGooglePay request.

### 2.2.4 Transaction Data

- **paymentType:** payment type, constant value **„CARD"**
- **total:** transaction total amount
- **currency:** currency code

### 2.2.5 Authorization Methods

When creating the Google Pay **paymentDataRequest** object, the merchant frontend must specify the supported authorization methods in the **allowedAuthMethods** field. The **allowedAuthMethods** field belongs exclusively to **PaymentDataRequest.allowedPaymentMethods[].parameters** in the Google Pay frontend configuration. It is not part of the SimplePay backend doGooglePay request.

**Example Google Pay paymentDataRequest configuration:**

```
//================================================================================
// Configuration
//================================================================================

// The DOM element that the Google Pay button will be rendered into
const GPAY_BUTTON_CONTAINER_ID = 'gpay-container';

// Update the `merchantId` and `merchantName` properties with your own values.
// Your real info is required when the environment is `PRODUCTION`.
const merchantInfo = {
  merchantId: simplePayTransaction.gpayMerchantId,
  merchantName: simplePayTransaction.gpayMerchantName
};

// This is the base configuration for all Google Pay payment data requests.
const baseGooglePayRequest = {
  /**
   * Define the version of the Google Pay API referenced when creating your
   * configuration
   *
   * @see {@link https://developers.google.com/pay/api/web/reference/request-objects#PaymentDataRequest
|apiVersion in PaymentDataRequest}
   */
  apiVersion: 2,
  apiVersionMinor: 0,
  allowedPaymentMethods: [
    {
      type: simplePayTransaction.paymentType,
      parameters: {
        /**
         * Card authentication methods supported by your site and your gateway
         *
         * @see {@link https://developers.google.com/pay/api/web/reference/request-objects#CardParamete
rs|CardParameters}
         * @todo confirm your processor supports Android device tokens for your
         * supported card networks
         */
        allowedAuthMethods: [
          "PAN_ONLY", "CRYPTOGRAM_3DS"
        ],
        /**
         * Card networks supported by your site and your gateway
         *
         * @see {@link https://developers.google.com/pay/api/web/reference/request-objects#CardParamete
rs|CardParameters}
         * @todo confirm card networks supported by your site and gateway
         */
        allowedCardNetworks: [
          "AMEX", "DISCOVER", "INTERAC", "JCB", "MASTERCARD", "VISA"
        ]
```

```
      },

      /**
      * Identify your gateway and your site's gateway merchant identifier
      *
      * The Google Pay API response will return an encrypted payment method capable
      * of being charged by a supported gateway after payer authorization
      *
      * @todo check with your gateway on the parameters to pass
      * @see {@link https://developers.google.com/pay/api/web/reference/request-objects#gateway|Payment
MethodTokenizationSpecification}
      */
      tokenizationSpecification: {
        type: 'PAYMENT_GATEWAY',
        parameters: {
          gateway: simplePayTransaction.gateway,
          gatewayMerchantId: simplePayTransaction.gatewayMerchantId
        }
      }
    }
  ],
  merchantInfo
};

// This is the endpoint of the merchant backend that receives data from frontend
// to convert it to SimplePay API request
//const authorizationEndpoint = "/YOUR_PAYMENT_SYSTEM_URL/dogooglepay.php";

// Prevent accidental edits to the base configuration. Mutations will be
// handled by cloning the config using deepCopy() and modifying the copy.
Object.freeze(baseGooglePayRequest);
```

### 2.2.6 Required Google Pay API Configuration and Initialization

The following examples illustrate the mandatory Google Pay Web API elements required for a compliant integration. In particular, the **paymentDataRequest** object must include the **transactionInfo** object, and the frontend implementation must verify Google Pay availability using the **isReadyToPay()** method before displaying the Google Pay button.

```
//================================================================================
// Helpers
//================================================================================

const deepCopy = (obj) => JSON.parse(JSON.stringify(obj));

function renderGooglePayButton() {
          const button = getGooglePaymentsClient().createButton({
                    onClick: onGooglePaymentButtonClicked
          });

          document.getElementById(GPAY_BUTTON_CONTAINER_ID).appendChild(button);
}

//================================================================================
// Event Handlers
//================================================================================

function onGooglePayLoaded() {
          const req = deepCopy(baseGooglePayRequest);
```

```
            getGooglePaymentsClient()
    .isReadyToPay(req)
    .then(function(res) {
                    if (res.result) {
                              renderGooglePayButton();
                              } else {
                              console.log("Google Pay is not ready for this user.");
                    }
            })
    .catch(console.error);
}

function onGooglePaymentButtonClicked() {
            // Create a new request data object for this request
            const req = {
                    ...deepCopy(baseGooglePayRequest),
                    transactionInfo: {
                              countryCode: 'EN',
                              currencyCode: simplePayTransaction.currency,
                              totalPriceStatus: 'FINAL',
                              totalPrice: simplePayTransaction.total
                    },
                    // todo: callbackIntents (codelab gpay-web-201)
            };

            // Write request object to console for debugging
            console.log(req);

            getGooglePaymentsClient()
            .loadPaymentData(req)
            .then(function (res) {
          // Write response object to console for debugging
          console.log(res);
          // @todo pass payment token to your gateway to process payment
          // @note DO NOT save the payment credentials for future transactions
          paymentToken = res.paymentMethodData.tokenizationData.token;
          console.error("paymentToken", paymentToken);

                    fetch(simplePayTransaction.authorizationEndpoint, {
                              method: "POST",
                              headers: {
                                        "Content-Type": "application/json",
                                        "Accept": "application/json"
                              },
                              body: JSON.stringify({
                                        total: simplePayTransaction.total,
                                        currency: simplePayTransaction.currency,
                                        paymentType: simplePayTransaction.paymentType,
                                        //googlePayToken: btoa(JSON.stringify(paymentToken))
                                        googlePayToken: btoa(paymentToken)
                              })
                    }).then(res => res.json()) // Parse the response as JSON.
                    .then(response => {
                              // 3DS challenge

                              const challengeRequired =
                              Array.isArray(response.errorCodes) &&
                              response.errorCodes.includes(3004) &&
                              typeof response.redirectUrl === "string";

                              if (challengeRequired) {
                                        // The browser leaves the merchant's payment initiation page.
                                        window.location.replace(response.redirectUrl);
```

```
                                        } else {
                                        // There are no SimplePay callbacks or automatic page reloads
.
                                        // The synchronous result is processed and displayed on this
page.
                                        console.log(response);

                                        // Optional, merchant-initiated navigation only:
                                        // window.location.replace(simplePayTransaction.notificationP
age);
                                }
                                }).catch(err => {
                                console.error("Error fetching authorization result", err);
                                alert(JSON.stringify(err))
                        });


            })
        .catch(console.error);
}
```

Optional navigation to notificationPage is initiated by the merchant frontend and is not a SimplePay back call. SimplePay redirects the browser to the merchant only after a 3DS challenge, using the url field of the backend request.

### 2.2.7 Billing Address Parameters

The Google Pay Web API allows merchants to optionally request the billing address associated with the selected payment card.

If required by the merchant's business logic or fraud prevention rules, the billing address may be requested using the **billingAddressParameters** object in the Google Pay payment data request.

The use of billing address parameters is optional and must follow Google Pay API recommendations.

If the billing address is collected on the frontend, it may also be forwarded to the merchant backend and included in the transaction request sent to the SimplePay API.

The assembled data is sent by the Google Pay JavaScript to the merchant backend together with the generated token. Based on this data, the merchant's system constructs the API call to be sent to the SimplePay backend **doGooglePay** endpoint.

### 2.2.8 Required fields sent from the frontend to the merchant backend:

Only the data that is actually required should be sent from the Frontend JavaScript to the merchant backend.

**Typically required fields include:**

- Google Pay payment token
- transaction amount
- currency code

- merchant transaction reference
- optional customer email

Data that can be generated on the merchant backend or is already stored there does not need to be exposed to the frontend. Examples include the customer email address, currency code, or the merchant's transaction reference. However, these may be stored in significantly different ways across merchant systems, so the actual system capabilities determine which data is exchanged between the frontend and the merchant backend.

The url used for returning from a 3DS challenge should not be accepted as an arbitrary frontend value. The merchant backend should add it to the doGooglePay request from trusted server-side configuration.

### 2.2.9 Google Pay Payment Backend, doGooglePay

The doGooglePay call is expected by the API at the following URL:

**https://sandbox.simplepay.hu/payment/v2/doGooglePay**

The SimplePay „**doGooglePay**" endpoint capabilities are fundamentally identical to those of the „**do**" endpoint. Therefore, only the differences are discussed below.

The following business logic features available on the „**do**" endpoint are not applicable on the „**doGooglePay**" endpoint:

- card-on-file (card storage) related features
- Event-specific back redirect parameters supported by the standard do endpoint. These are distinct from the url field of the doGooglePay request, which specifies the browser return destination used exclusively after a 3DS challenge. It must be an absolute HTTPS URL supplied by the merchant backend from trusted server-side configuration. SimplePay does not navigate to this URL if no 3DS challenge occurs.

The Google Pay payment token must be included in the request as the string field **googlePayToken**.

**Sample JSON Request**


```
{
          "salt":"cb9847b1667b69bace46b64ac10d8672",
          "merchant":"TESTMERCHANT",
          "googlePayToken":"eyJzaWduYXR1cmUiOiJNRVVDSVFDbWw5djFHUms0NkE0QzltSC9tR1c0MENwS3d2ME1FeGhKbT
ROUDdQZys0Z0lnZllFQWszWVpwamc4d0NQaGNQSDNmREY2UGtkcnBZOHNRc3l4VkVPMTNMc1x1MDAzZCIsImludGVybWVkaWF0ZVNpZ
25pbmdLZXkiOnsic2lnbmVkS2V5Ijoie1wia2V5VmFsdWVcIjpcIk1Ga3dFd1lIS29aSXpqMENBUVlJS29aSXpqMERBUWNEUWdBRWNS
NmFBaHZwOHJjZUN6RUF6dTZpd0xmTXJyV0RpVTVYdWlkYVVrMlhISkE1Z0dvaXNEU1dMcTUvRnRSRXY3akV1NFkzQjAyTy9scVRFaUF
1eHZOQ0pRXFx1MDAzZFxcdTAwM2RcIixcImtleUV4cGlyYXRpb25cIjpcIjE3Njk3NjkwNjMyMjNcIn0iLCJzaWduYXR1cmVzIjpbIk
1FUUNJQXc5TFNDK3ppeHFDN2l5eHEreUF6aG9KRWVpZWZla3RkZ20yZ1lMbU9CMEFpQkp1YUFzODhDNVkyZkNTbDN6a2FrbEtFSmw5O
```

XgyNXpWR0pienVJQ0ZFTWdcdTAwM2RcdTAwM2QiXX0sInByb3RvY29sVmVyc2lvbiI6IkVDdjIiLCJzaWduZWRNZXNzYWdlIjoie1wi
ZW5jcnlwdGVkTWVzc2FnZVwiOlwiRkxPOGlLSWx4elZLVkRuS21ZcEkxbHFNSFhLZjFxRkx4Nmp0cGJkSG9raTB1aUh0NEhzajd6bUl
MQ2JsQW9nRmwvQmw0NWZ5L2RHODRSNUtyUWRMRjVMLzVEVEhWZ2N5TWp2K2pJV2hEYUZVb0Fud0lWMFdnL0RyTmppdVpxV0lGUmZRV1
dDbGJjTC9CK05LR21Ib2tranlkbCtteUpSeVNjM1JVbFYydklvUW9MOFBWZngzOXY4aDZrUEdHcFBzY0dhVHlTdmsvRmUvZkNlY05RS
mpDNDFrRVFmY1h3QmxyUkFoRnVKcmpFUlRUZkZPYUhob1pCRmNiWE85SzJLTUpERW1BMDA3Q1VVbkJFNVFhN2doTWF4VVdpZ3JqclFR
NVZkMWtMbGtBMGQ0YTdxa1B0THZUYTRFLytIVys1OElTaHBoNDRRYldGdmIwRXVQVmF6TjlzaEovZVdDM0pnQkExWnNPblp2T0NONSs
vZEdQSXoyV0lpM0FiQjRrY0VzeU9rUlN3Y1hnM2lMeWtMYnBBZVhvRWtPbVpOZ1pqcW9oUnlGdjRHaHNoYmFyVXdqZTk0K0JRRjVOST
IvTFpkTjNSdXB4OGRBTzBEa3hkUUZxc1h5UmpZdmQ5RmRjTmV2ekM5QlVyL2hWUHlqTzVXZGFjcGFUVWZFTUs2NkRXTUZwU2dRTXdye
nZWcjBMUVAvL2Z6QzFsL0UwallaeDNKMVZOUTBwejhLUEJCc3VjRGd4bEh6Mi91bWcyTkc5bC90cENBK2RNM2NIdFM2SzhnXFx1MDAz
ZFwiLFwiZXBoZW1lcmFsUHVibGljS2V5XCI6XCJCSHBxcUt2RVlvcUtqUkl2ak1JMDYzc2hBZWFOUjdPZlBCMEZsRjR0VEhzZ2E1L1p
UOUw2ZDJ0dzZWWmd5TnBVT2FINzZkYTdudXFsdnh4MGRPM0c3N1FcXHUwMDNkXCIsXCJ0YWdcIjpcImxXeGpwN1pleVk5U1JKdyt3WW
VrSG9jZUhhMnNVQWZFcVlRNnFaVzIzTzhcXHUwMDNkXCJ9In0=",
"sdkVersion":"SimplePay_PHP_SDK_2.1.5_251214:1eb306c95cfefb4f5252f0dbde9704ef",
"currency":"HUF",
"total":"103",
"type":"CIT",
"orderRef":"10891217690904437535",
"url": "https://sdk.simplepay.hu/googlepay/back"
"customerEmail":"sdk_test@simplepay.com",
"language":"EN",
"timeout":"2026-01-23T14:00:43+00:00"
}

### Sample JSON Response

The response for a successful transaction is identical to that of a standard card payment.

If the issuing bank requires 3DS authentication (Strong Customer Authentication), the errorCodes array will contain error code 3004. In this case, the redirectUrl in the response contains the URL to which the browser must be redirected. The customer can then complete the standard Strong Customer Authentication process.

{
"errorCodes":[
3004
],
"merchant":"TESTMERCHANT",
"salt":"0IhfDe1so9WhtwUwdekpccdClzDAHQQp",
"transactionId":10312754,
"orderRef":"10891217690904437535",
"currency":"HUF",
"total":103.0,
"redirectUrl":"https://sb-checkout.simplepay.hu/pay/pay/pspHU/0RNkfXmcwVO9KqpD6o1edrGDstWBcgIwySJ43ZuDA
DcevUUXDV"
}

The redirectUrl is used only to start the 3DS authentication flow. After the challenge has been completed, SimplePay redirects the customer to the URL specified in the request's url field.

If the transaction fails for any reason, the „**errorCodes**" field in the response will contain the error code.

The synchronous response from the „**doGooglePay**” endpoint is received by the merchant API, which must then return the required data to the frontend JavaScript.

If the issuing bank requires 3DS Strong Customer Authentication (SCA), the JavaScript automatically redirects the customer to the appropriate SimplePay page where the authentication can be completed.

**Non-3DS response**

If no 3DS challenge is required, the merchant backend returns the synchronous doGooglePay result to the frontend. The customer remains on the payment initiation page. SimplePay does not initiate a page reload, browser redirect, or back call.

**Post-3DS back call**

After the 3DS authentication has been completed, SimplePay redirects the browser to the URL specified in the request's url field. The request contains the r and s parameters. The r parameter contains the Base64-encoded result and s contains the SimplePay signature. The result must not be processed until the signature has been successfully verified.

To enable and support **3DS PAN_ONLY** credentials, the **PAN_ONLY** and **CRYPTOGRAM_3DS** values must be configured in the frontend Google Pay **PaymentDataRequest.allowedPaymentMethods[].parameters.allowedAuthMethods** field. The **allowedAuthMethods** field is not part of the backend doGooglePay request. The related sample code can be found in 2.2.5. Authorization method example.

### 2.2.10 Invoking the Google Pay payment sheet

When the customer presses the Google Pay button, the merchant frontend must call the loadPaymentData() method of the Google Pay API.

**Example:**

```
paymentsClient.loadPaymentData(paymentDataRequest)
.then(function(paymentData) {

    const token = paymentData.paymentMethodData.tokenizationData.token;

    // token továbbítása a backend felé
    fetch(simplePayTransaction.authorizationEndpoint, {
                    method: "POST",
                    body: JSON.stringify({
                              googlePayToken: btoa(token),
                              total: simplePayTransaction.total,
                              currency: simplePayTransaction.currency
                    })
          });

})
.catch(function(err) {
    console.error(err);
```

```
});
```

The response must be handled using the two-branch logic described in Section 2.2.6. If error code 3004 and a redirectUrl are returned, the browser is redirected to the 3DS flow. Otherwise, the synchronous result is processed on the payment initiation page.

## 2.3 Supported Payment Methods

During the payment flow, the Google Pay interface supports card payments using multiple card networks and authorization methods.

Supported card networks include: **Visa** / **Mastercard** / **American Express** (depending on the merchant's acquiring agreement)

**The availability of specific card networks may depend on the merchant's contractual relationship with the acquiring bank or payment service provider.**

Within the Google Pay Web API, the following authorization methods are supported:

**PAN_ONLY** – card details are provided in tokenized form without device-level cryptography. Depending on the issuing bank and regulatory requirements, additional Strong Customer Authentication (SCA) such as 3DS may be required.

**CRYPTOGRAM_3DS** – device tokens generated by the Google Pay ecosystem that include a dynamic cryptogram for authentication. These transactions may be processed without additional authentication depending on the issuer's risk assessment.

**Both authorization methods are supported by the SimplePay Google Pay acceptance flow and can be processed through the SimplePay API.**

# 3 Merchant Requirements Mandated by Google

## 3.1 Business Compliance

To accept Google Pay, the merchant must be a legally operating business. Google performs a business compliance review when enabling the payment method, which includes:

- a legally registered business entity
- verifiable and authentic company information
- transparent business operations
- legally distributed products and services

It is the merchant's responsibility to ensure that the products and services offered comply with Google Pay's business policies.

## 3.2 Prohibited and Restricted Activities

Google Pay cannot be used for the sale of products or services that are illegal, deceptive, or in violation of Google's Acceptable Use Policy.

The scope of prohibited or restricted activities is determined by Google, and compliance is the merchant's responsibility. If an activity does not meet these guidelines, the Google Pay payment method cannot be enabled.

# 4 Google Pay Approval-Related Elements

## 4.1 Google Pay Merchant Identification

In the Google Pay system, every transaction is associated with a Google-issued merchant identifier (**gpayMerchantId**).

When accepting payments via the SimplePay service:

- Although the merchant does not need to establish a direct backend integration with Google Pay, a frontend JavaScript integration using the Google Pay Web API is still required in the merchant's checkout interface.
- This frontend integration is responsible for displaying the Google Pay button, invoking the Google Pay payment sheet, and retrieving the payment token that is subsequently forwarded to the merchant backend and the SimplePay API.
- Providing the business data required by Google is mandatory.

Google's approval is contingent upon the availability and acceptance of the appropriate business data.

### 4.1.1 Obtaining the gpayMerchantId (Google Pay merchantId) – Creating a Merchant Profile (Google Pay & Wallet Console)

For production Google Pay web acceptance, the merchantId associated with the merchant profile created in the Google Pay & Wallet Console is required (referred to as merchantInfo.merchantId in the documentation). The Google identifier appears on the console interface after profile creation.

1. Sign in to the Google Pay & Wallet Console.
2. Under „Type of business", select **Merchant** and provide the required basic information.
3. Complete **the Business Profile** (company details).
4. In the left-hand menu, select the **Google Pay API** tab, then click **Get Started**.

5. Accept the **Google Pay API Terms of Service** and the **Acceptable Use Policy**.
6. The merchantId then appears in the upper-right area of the interface (Google indicates this in the step list).

The result of the above process is the merchant's own **gpayMerchantId/merchantId**.

## 4.2 Domain and Application Compliance

Google Pay can only be used on online interfaces that meet the following requirements:

- a website secured with the HTTPS protocol, or
- a live, publicly available mobile application

Google may verify the compliance of the domain or application operated by the merchant. Technical registration and payment feature enablement are handled through the SimplePay system.

### 4.2.1 Domain verification

Before Google Pay can be used in the PRODUCTION environment, the merchant domain must be registered and approved in the Google Pay & Wallet Console.

**The domain must:**

- use HTTPS
- be publicly accessible
- host the Google Pay button within the checkout flow

Google reviews the submitted integration screenshots and verifies that the Google Pay button and payment flow comply with the integration checklist.

### 4.2.2 Submitting a Website (Domain) and Requesting Approval (for Production Use)

Google requires that the website integration be approved before production use:

1. The merchant verifies that the integration meets the requirements of Google's „Integration checklist".
2. Console: Google Pay API → Integrations → „Integrate with your website" → **Add website**.
3. Enter the **top-level domain** that calls Google Pay.
4. **Integration type: select Gateway** (SimplePay model, i.e., via a payment service provider).

5. Upload screenshots demonstrating the Google Pay integration in the checkout flow.
6. Save, then **Submit for approval**.

## 4.3 What We Require from the Merchant Regarding the merchantId

Under the SimplePay acceptance model, the merchant must provide the following data/artifacts to SimplePay:

- the merchant's **gpayMerchantId/merchantId** (from the Console)
- the production domain(s) and the top-level domain submitted to Google
- screenshots used for Google approval (checkout steps)
- a declaration/confirmation that the functional and brand requirements specified in the integration checklist are met (in particular: isReadyToPay(), loadPaymentData() behavior, parity, and total amount)

### 4.3.1 Submission process

The required screenshots must be submitted to SimplePay through the official merchant support channel.

This may include:

- the SimplePay merchant support ticket system
- the technical contact email provided during merchant onboarding

The screenshots must demonstrate the checkout flow and the placement of the Google Pay button.

# 5 Technical and System-Level Elements Provided by SimplePay

When implementing Google Pay acceptance via the SimplePay service, **the following areas fall within SimplePay's scope**:

- payment API interface provisioning
- technical integration with the merchant's system
- processing of Google Pay payment tokens
- transaction management and status reporting
- management of system identifiers and technical keys

The merchant connects to the service based on the documentation and interfaces provided by SimplePay, without the need to establish a direct Google Pay API connection.

For mobile application integrations, developers should refer to the official Google Pay Android documentation:

**Google Pay Android Developer Documentation:**
https://developers.google.com/pay/api/android/overview

**Google Pay Android Integration Checklist:**
https://developers.google.com/pay/api/android/guides/test-and-deploy/integration-checklist

**Google Pay Android Brand Guidelines:**
https://developers.google.com/pay/api/android/guides/brand-guidelines

# 6 Testing and Production

Testing the payment flow is mandatory before enabling Google Pay acceptance. The following is verified during testing:

- end-to-end execution of the payment flow
- handling of both successful and failed transactions
- display of payment result notifications to the customer
- successful payment without 3DS, while remaining on the payment initiation page
- failed payment without 3DS
- response containing error code 3004 and a redirectUrl
- successful 3DS authentication and return to the configured url
- failed, cancelled, and expired 3DS authentication
- missing or invalid r and s parameters
- different arrival orders of the browser back call and IPN
- customer closing the browser before returning to the merchant

The testing and go-live process is handled through SimplePay's systems. Google Pay usage is only permitted in an approved state.

# 7 Summary

Google Pay acceptance via SimplePay operates according to the following principles:

- business and legal compliance required for Google Pay usage is verified by Google
- technical integration and API connectivity are provided through the SimplePay service
- The merchant does not need to establish a direct backend integration with Google Pay. However, a frontend JavaScript integration using the Google Pay Web API is required in the checkout interface.
- the payment flow is processed in a secure, tokenized manner

Under normal conditions, the Google Pay flow remains on the merchant's payment initiation page. A SimplePay-initiated browser back call occurs only when the customer was previously redirected to the SimplePay interface for a 3DS challenge.

The above structure ensures that the Google Pay payment method is deployed in a legally compliant, secure manner, in accordance with Google's guidelines, through the service provided by SimplePay.